Guía de estudio: de Monte Carlo a DQN y A2C

Con el robot balanceador como ejemplo de punta a punta

Material de apoyo personal para el Desafío 1 — no es parte de la entrega

0. El mapa: por qué aparecieron estos algoritmos

Cada algoritmo nuevo resuelve una limitación concreta del anterior. La intuición de que se derivaron "para pasar de discreto a continuo" es **en parte correcta**: el motivo principal de DQN es que el **estado** sea continuo (no entra en una tabla), y uno de los motivos de A2C es poder manejar **acciones** continuas (aunque acá se usó con acciones discretas). Hay otros dos motivos igual de importantes: no tener que esperar al final del episodio para aprender, y bajar el ruido de las estimaciones.

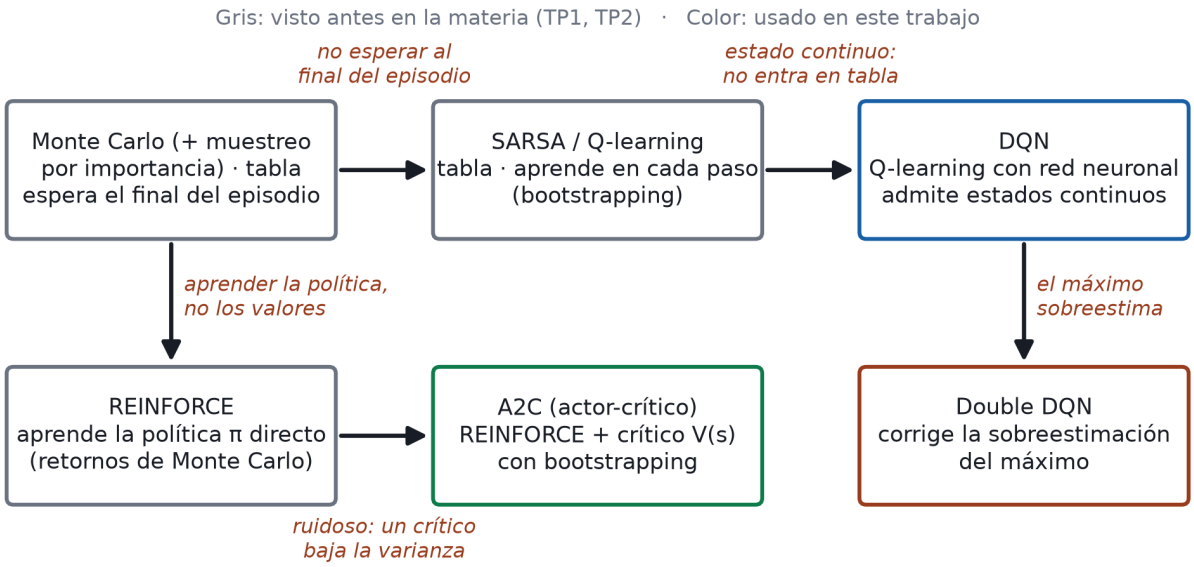


Figura 1 — Qué problema resuelve cada paso. La fila de arriba aprende valores; la de abajo, la política.

1. Punto de partida: lo que ya sabés de Monte Carlo

El objetivo de siempre: encontrar una **política** π (qué acción tomar en cada estado) que maximice el **retorno** G, la suma descontada de recompensas futuras:

$$G_t = r_{t+1} + \gamma\, r_{t+2} + \gamma^2\, r_{t+3} + \cdots$$

Monte Carlo estima Q(s, a), el retorno esperado si en s se toma a y después se sigue π, **promediando retornos reales** de episodios completos. Con **muestreo por importancia** se puede aprender de una política (la objetivo) mientras se juega con otra (la de comportamiento), corrigiendo cada retorno por el cociente de probabilidades de las dos.

Sus dos limitaciones son las que motivan todo lo que sigue:

- **Hay que esperar al final del episodio** para conocer G. Si el episodio es largo (500 pasos acá), el aprendizaje es lento, y los retornos de episodios largos varían mucho (ruido).
- **Una casilla de la tabla por cada par (estado, acción).** Sirve si los estados son pocos y discretos (FrozenLake: 16). El robot tiene estados continuos: θ puede valer 1.2345°.

2. SARSA y Q-learning: aprender en cada paso

La idea de **diferencias temporales (TD)**: no esperar a G, sino estimarlo con la recompensa del paso más lo que la propia tabla dice que vale el estado siguiente. Eso se llama **bootstrapping**: usar una estimación para mejorar otra. **Q-learning** actualiza así:

$$Q(s,a) \leftarrow Q(s,a) + \alpha\left[r + \gamma \max_{a'} Q(s',a') - Q(s,a)\right]$$

Lo que está entre corchetes es el **error TD**: la diferencia entre lo que se esperaba, Q(s, a), y una estimación un poco mejor, r + γ·max Q(s', a'). Q-learning es **off-policy** como el Monte Carlo con muestreo por importancia, pero **sin cocientes**: el max ya evalúa la política golosa, sin importar qué acción se haya jugado de verdad.

La limitación que queda: sigue siendo una tabla. Se podría dividir θ, dθ/dt y u en 20 intervalos cada uno (8 000 estados × 9 acciones = 72 000 casillas), pero cada casilla aprendería sola: lo aprendido en θ = 1.0° no ayudaría en θ = 1.1°. Hace falta algo que **generalice** entre estados parecidos.

3. DQN: Q-learning con una red neuronal

DQN reemplaza la tabla por una **red neuronal** que recibe el estado y devuelve un Q por acción. Estados parecidos producen salidas parecidas: la red generaliza. La actualización es la misma idea que Q-learning, pero en vez de corregir una casilla se ajustan los pesos de la red para achicar el error entre Q(s, a) y el objetivo y = r + γ·max Q(s', a').

La red usada: 3 → 64 → 64 → 9

- **3 entradas**: las variables del estado (θ, dθ/dt, u anterior), normalizadas.
- **9 salidas**: un Q por acción. Con una sola pasada se obtienen los nueve valores y se elige el mayor. (La alternativa, meter la acción como entrada, obligaría a 9 pasadas.)
- **Dos capas ocultas de 64 neuronas**: tamaño habitual para control con pocas variables (CartPole en clase). Alcanza para representar la función y entrena rápido; no se optimizó. En total, unos 5 000 pesos (3·64+64 + 64·64+64 + 64·9+9 = 5 001).

Los tres "trucos" que la hacen estable

Combinar Q-learning con una red tiende a oscilar o diverger. DQN agrega:

Truco	Qué hace	Qué problema evita
Exploración ε-greedy	con probabilidad ε juega una acción al azar; ε baja de 1 a 0.02	sin explorar, nunca descubre que otra acción era mejor
Replay buffer	guarda las últimas 50 000 transiciones (s, a, r, s', cayó) y entrena con lotes de 64 sacados al azar	pasos seguidos son casi idénticos: entrenar con ellos en orden es como estudiar repitiendo la misma página. Además reutiliza cada experiencia muchas veces
Red objetivo	copia congelada de la red, actualizada cada 1 000 pasos, que calcula y	si y se calculara con la misma red que se está ajustando, el blanco se movería con cada corrección ("perseguirse la cola")

Por qué DQN necesita acciones discretas: elige la acción con max sobre las salidas de la red. Con acciones continuas (cualquier duty entre −121 y +121) habría infinitas y no se puede tomar el máximo así. Por eso el robot tiene 9 niveles de duty.

4. Double DQN: corregir el optimismo del máximo

Las estimaciones de Q tienen ruido. Tomar el **máximo de valores ruidosos** da, en promedio, más que el verdadero máximo. Ejemplo: si las 9 acciones valen en realidad 50, pero la red las estima con un error de ±2 al azar, el máximo de las nueve estimaciones sale ≈ 53. Ese optimismo se copia en el objetivo y y se acumula paso a paso.

Double DQN separa elegir de evaluar: la red principal **elige** la mejor acción en s', y la red objetivo (con errores distintos) dice **cuánto vale**. Es un cambio de una línea en el código:

$$\text{DQN}: \quad y = r + \gamma \max_{a'} Q_{obj}(s', a')$$

$$\text{Double DQN}: \quad y = r + \gamma\, Q_{obj}\left(s',\ \operatorname{argmax}_{a'} Q(s', a')\right)$$

En el robot no mejoró nada (con 10 semillas fue incluso apenas más lento): con 9 acciones y recompensas entre 0 y 1, el optimismo es chico. Double DQN brilla con muchas acciones y recompensas ruidosas (por ejemplo, Atari).

5. De REINFORCE a A2C: aprender la política directamente

Los métodos anteriores aprenden **valores** y la política sale de elegir el mayor. Los métodos de **gradiente de política** aprenden **la política misma**: una red que da la probabilidad de cada acción, π(a|s). Ventajas: funcionan igual con acciones continuas (la red da, por ejemplo, la media y el desvío de una gaussiana) y la política puede ser aleatoria cuando conviene.

REINFORCE (Monte Carlo para políticas)

Juega un episodio completo y, para cada acción tomada, **sube su probabilidad en proporción al retorno G** que siguió. Si G fue alto, la acción se vuelve más probable. Es Monte Carlo: hereda su problema de **ruido**, porque G depende de todo lo que pasó después, no solo de esa acción.

A2C: agregar un crítico

A2C tiene dos redes. El **actor** es la política π(a|s). El **crítico** estima V(s), cuánto se espera ganar desde s. En vez de usar G, el actor usa la **ventaja**:

$$A = r + \gamma V(s') - V(s)$$

"¿Me fue mejor o peor de lo que el crítico esperaba?". Si A > 0 se sube la probabilidad de la acción; si A < 0, se baja. Comparar contra lo esperado (en lugar de contra cero) y usar V(s') en vez de esperar al final reduce mucho el ruido. GAE(λ) es una forma de promediar ventajas de 1, 2, 3… pasos para equilibrar sesgo y ruido.

	DQN / Double DQN	A2C
Aprende	Q(s, a): valor de cada acción	π(a|s) y V(s)
Política	elegir el mayor Q (determinista)	la red da probabilidades (aleatoria)
Datos	**off-policy**: reutiliza experiencia vieja (replay)	**on-policy**: solo lo recién jugado
Datos no correlacionados con	replay buffer	8 entornos en paralelo
Explora con	ε-greedy	su propio azar + bonus de entropía
Acciones continuas	no	sí (con otra salida de la red)

Consecuencia práctica, visible en los resultados: DQN necesitó menos de la mitad de experiencia que A2C, porque reutiliza cada transición muchas veces; A2C la usa una sola vez y la descarta.

6. El robot como MDP, en detalle

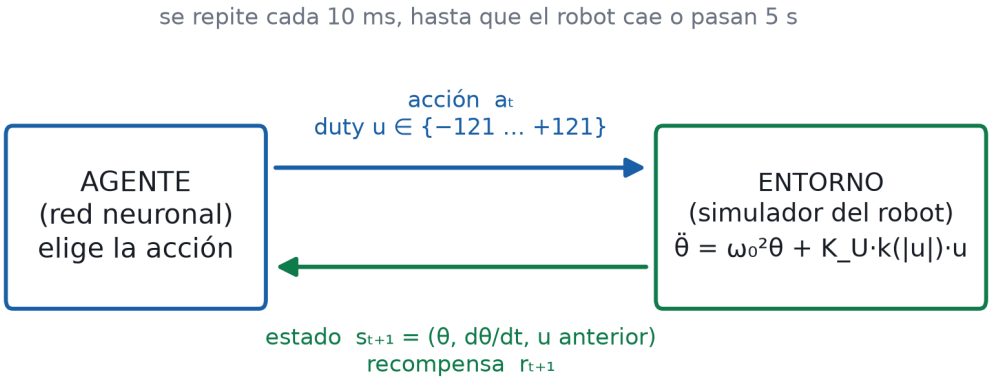


Figura 2 — Lo que pasa cada 10 ms.

Estado: qué ve el agente

Tres números: θ (inclinación), dθ/dt (qué tan rápido cambia la inclinación) y el último duty aplicado. θ solo no alcanza: con θ = 2° el robot puede estar cayendo o volviendo, y la acción correcta es distinta; dθ/dt lo distingue. Antes de entrar a la red se **normalizan** dividiendo por una escala típica (θ entre 10°, dθ/dt entre 1.4 rad/s, u entre 121) para que los tres queden entre −1 y 1 aproximadamente: si no, el duty (≈ 100) dominaría a θ (≈ 0.05 rad) y la red casi ignoraría la inclinación.

Acciones: por qué 9 niveles

DQN necesita una lista finita (sección 3). Los niveles son 0, ±75, ±90, ±105 y ±121 de duty. No hay niveles chicos (por ejemplo 30) porque por debajo de 66 el motor no se mueve: serían acciones inútiles. Esa ausencia de "empujes finos" es la que hace que el robot quede pulsando cerca de 0°.

Recompensa: qué se premia

$$r = 1 - \left(\frac{\theta}{10^\circ}\right)^2 - 0.05\left(\frac{u}{121}\right)^2 - 0.05\left(\frac{\Delta u}{242}\right)^2$$

Situación	r aproximada	Por qué
derecho (θ = 0°), sin motor	1.00	lo ideal
θ = 3°, sin motor	0.91	$3^2/10^2 = 0.09$ de castigo
θ = 0°, motor a fondo sin cambiarlo	0.95	$0.05 \cdot 1^2$ de castigo por energía
θ = 0°, pasar de −121 a +121	0.90	gasto + cambio brusco
θ = 9.5°	0.10	casi cayendo: vale casi nada
cayó (|θ| > 10°)	0 y fin	no hay más recompensas

Así el agente aprende, en orden de importancia: no caer, estar derecho y, si puede, no gastar ni vibrar. Los pesos 0.05 son chicos a propósito para que nunca compitan con no caer.

Estado inicial: cómo se apoya el robot

Cada episodio simula apoyar el robot a mano "lo más vertical que el ojo permite". Ese error se midió en el robot real: 22 apoyos, registrando la inclinación en el momento en que quedaba quieto (velocidad angular < 0.15 rad/s). Promedio 1.98°, peor caso 3.06°. Por eso el episodio arranca con **dos** valores al azar: un ángulo entre −3.06° y +3.06° y una velocidad angular entre −0.15 y +0.15 rad/s (la mano nunca lo suelta perfectamente quieto). Son variables distintas, por eso tienen unidades distintas.

El descuento γ = 0.99: miopía controlada

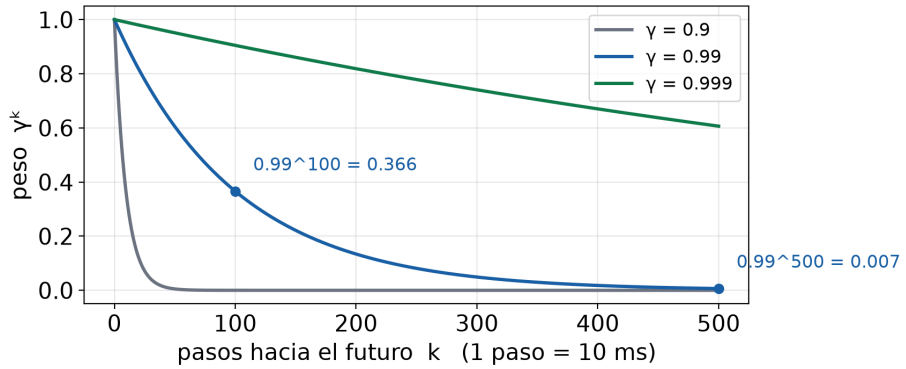


Figura 3 — Cuánto pesa hoy una recompensa que llega k pasos más adelante.

γ no hace que las recompensas sean más chicas: cada paso de pie sigue valiendo ≈ 1. Lo que hace es que, **al decidir**, el agente valore menos lo lejano. Con 0.99, lo que pasa dentro de 1 s pesa bastante (más de 0.37); lo que pasa a 5 s, casi nada (0.007). Como sin control el robot cae en 0.4 s, mirar ≈ 1 s hacia adelante es suficiente. Con γ = 0.9 miraría solo ≈ 0.1 s, demasiado corto para anticipar; con 0.999, ≈ 10 s, y el aprendizaje sería más lento y ruidoso.

Por eso hay dos escalas de "retorno": el **sin descuento** (suma simple de las recompensas, máximo ≈ 500 = 500 pasos × 1, el de los gráficos) y el **descontado** que estima Q (máximo ≈ $1/(1-\gamma) = 100$). Una política al azar cae en ≈ 35 pasos sumando ≈ 0.85 por paso: ≈ 30.

Un episodio real, paso a paso

Así decide la DQN ya entrenada, partiendo de θ = 3° y quieto. En cada fila, el agente ve el estado, su red calcula los 9 valores Q, elige el mayor, el simulador avanza 10 ms y devuelve r:

t	θ	dθ/dt	estado normalizado	duty	r	Q elegida	Q(duty 0)
0 ms	3.00°	+0.000	(+0.30, +0.00, +0.00)	-75	0.886	77.3	77.3
10 ms	3.00°	+0.001	(+0.30, +0.00, -0.62)	-121	0.858	77.2	77.1
20 ms	3.00°	-0.017	(+0.30, -0.01, -1.00)	-121	0.861	77.4	77.1
30 ms	2.98°	-0.036	(+0.30, -0.03, -1.00)	-121	0.863	77.5	77.3
40 ms	2.95°	-0.055	(+0.30, -0.04, -1.00)	-121	0.865	77.6	77.5
50 ms	2.92°	-0.074	(+0.29, -0.05, -1.00)	-105	0.880	77.8	77.6

t	θ	dθ/dt	estado normalizado	duty	r	Q elegida	Q(duty 0)
60 ms	2.87°	-0.087	(+0.29, -0.06, -0.87)	-121	0.871	77.9	77.7
70 ms	2.81°	-0.107	(+0.28, -0.08, -1.00)	-121	0.875	78.0	77.9
80 ms	2.74°	-0.128	(+0.27, -0.09, -1.00)	-121	0.879	78.1	78.0
90 ms	2.66°	-0.150	(+0.27, -0.11, -1.00)	-121	0.884	78.2	78.1

Cómo leerla: el estado normalizado es lo que entra a la red. **Q elegida** es el retorno descontado que la red espera si toma esa acción y después sigue jugando bien. **Q(duty 0)** es lo que espera si en ese paso no hiciera nada. La diferencia es chica (0.0 a 0.2) y eso tiene sentido: no actuar durante un solo paso de 10 ms se puede corregir en el siguiente, y Q supone que después se juega bien. Para decidir no importa el valor exacto de Q sino **cuál acción tiene el mayor**.

Dos detalles más. Los Q rondan 77, por debajo del máximo teórico de 100, porque descuentan las inclinaciones y el gasto de motor futuros y porque son una estimación aproximada: a la red le alcanza con ordenar bien las acciones. Y la recompensa r es casi constante (≈ 0.87) en todos los pasos: el aprendizaje no está en r sino en Q, que resume todo el futuro.

7. Cómo leer los resultados del informe

- **Curva de convergencia**: retorno por episodio a lo largo del entrenamiento, suavizado. Empieza en ≈ 30 (cae enseguida) y sube hacia 500 (aguanta los 5 s).
- **Contra episodios o contra pasos**: A2C juega muchos episodios cortos en paralelo, así que por episodio parece más lento de lo que es. Contra pasos de entorno se compara cuánta experiencia necesita cada uno.
- **Banda alrededor de la curva**: ± un desvío entre semillas. Si las bandas de dos algoritmos se superponen mucho, la diferencia puede ser azar.
- **Evaluación sin exploración**: lo que importa al final. Se apaga el azar (ε = 0 en DQN; acción más probable en A2C) y se juega en episodios nunca vistos.
- **Semillas**: el mismo algoritmo puede salir mejor o peor según el azar de la inicialización y la exploración. Por eso se repite con varias.

8. Glosario mínimo

Término	Significado
Episodio	un intento completo: desde que se apoya el robot hasta que cae o pasan 5 s
Retorno G	suma (descontada) de las recompensas desde un momento hasta el final
Q(s, a)	retorno esperado si en el estado s se toma la acción a y después se juega bien
V(s)	retorno esperado desde s siguiendo la política (el promedio de Q sobre las acciones)
Bootstrapping	estimar el futuro con la propia estimación (V o Q del estado siguiente)
On / off-policy	aprende solo de la política que juega / puede aprender de otra (o de datos viejos)
Ventaja A	cuánto mejor resultó una acción respecto de lo esperado: r + γV(s') − V(s)
Terminal / truncado	el episodio terminó por caer (no hay futuro) / se cortó por tiempo (sí lo habría)
Semilla	número que fija el azar; distinta semilla = distinta corrida del mismo algoritmo

Para profundizar: Sutton y Barto (2018), capítulos 5 (Monte Carlo), 6 (TD, Q-learning), 9 (aproximación de funciones) y 13 (gradiente de política y actor-crítico); Mnih et al. (2015) para DQN. Y el notebook `guia_estudio.ipynb`, que tiene celdas para probar cada idea.